



ADITYA UNIVERSITY

SCHOOL OF ENGINEERING

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Cornerstone project work entitled “**DIGITAL LIBRARY SYSTEM**” is submitted by

K. Madhu Kiran	(24B11CS182)
K. Teja Siri	(24B11CS185)
K. Rakshitha	(24B11CS188)
K. Swathi	(24B11CS175)

in partial fulfillment of the requirements for the award of the B.Tech. degree in Computer Science and Engineering for the academic year 2025-2026.

Project Guide

Mr.Ch. Bhanu Prakash,M.tech (Ph.D)
Assistant Professor,
Department of CSE

Head of the Department

Dr. T.Sudha Rani, Ph.D,
Assistant Professor,
Department of CSE

DECLARATION

We hereby declare that the Cornerstone project entitled “**DIGITAL LIBRARY SYSTEM**” is original and is submitted to **ADITYA UNIVERSITY**, Surampalem, in partial fulfillment of the requirements for the award of the **B.Tech. degree in Computer Science and Engineering**. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy

Place: Surampalem

Date:

By

K. Madhu Kiran	(24B11CS182)
K. Teja Siri	(24B11CS185)
K. Rakshitha	(24B11CS188)
K. Swathi	(24B11CS175)

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, **Mr.Ch. Bhanu Prakash, Assistant Professor**, who has guided us throughout the project. His valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, **Dr. Hari Jyothula Associate Professor and Dr.K.Chandra Shekar, Associate professor**, Department of CSE, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department ,**Dr.T.Sudha Rani, Assistant Professor**, for his inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean, School of Engineering** for their encouragement and support during the course of our project.

We would like to express our sincere gratitude to **Dr.G.Suresh,Registrar, Dr.A Ramesh, ProVice-Chancellor(Engineering &Sciences), Dr.S.Rama Sree,ProVice-Chancellor (Academics), Dr.M.B.Srinivas, Vice-Chancellor, Dr. M.Sreenivasa Reddy, Deputy Pro Chancellor and Management, Aditya University** for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **faculty members, lab technicians, non-teaching staff, and our friends** who have directly or indirectly supported us in completing this Cornerstone project successfully.

TABLE OF CONTENTS

Description	Page No.
Abstract	i
List of Figures	ii
List of Tables	iii
Acronyms	iv
1. INTRODUCTION	1
1.1 Brief Information about the Project	1
1.2 Motivation and Contribution of the Project	1
1.3 Objectives of the Project	1
1.4 Scope of the Project	2
2. SYSTEM ANALYSIS	3
2.1 Functional Requirements	3
2.2 Non-Functional Requirements	3
2.3 Requirements Specification	4
3. TECHNOLOGY DESCRIPTION	5
3.1 Programming Languages	5
3.2 Frameworks and Libraries	5
3.3 Database Technology	6
3.4 Tools and IDEs	6
4. DATABASE DESIGN	7
4.1 Database Tables (Collections)	7
4.2 Collection Structure	7
4.3 Keys and Constraints	8
4.4 Collection Relationships	8
5. SYSTEM DESIGN	9
5.1 System Architecture	9
5.2 Module Descriptions	10
6. SYSTEM IMPLEMENTATION	11
7. CONCLUSION	15
8. FUTURE ENHANCEMENTS	16
9. REFERENCES	17
10. Appendix	18

ABSTRACT

The Digital Library System is a comprehensive, full-stack web application developed using the MERN stack (MongoDB, Express.js, React, Node.js) to modernize and streamline library operations. This system moves away from traditional, manual cataloging processes by offering a dynamic, centralized digital platform where users can easily register, log in, browse, search, and download digital books securely. With an emphasis on user experience, the frontend is built using React 19 and Vite to ensure high performance and a highly responsive interface.

Security and structured access control are pivotal aspects of this platform. The backend utilizes Express.js and Node.js, implementing JWT (JSON Web Tokens) for robust session management and bcrypt for secure password hashing. A role-based access control system clearly separates standard Users from Administrators. While Users can search the catalog, filter by categories, and manage their reading carts, Administrators possess full CRUD (Create, Read, Update, Delete) privileges to maintain the book inventory. MongoDB, along with the Mongoose ODM, is employed for scalable and flexible data storage, making the Digital Library System a scalable, secure, and modern approach to digital resource management.

LIST OF FIGURES

Figure No.	Figure Name	Page No.
6.1	Sign Up Page	11
6.2	Login Page	12
6.3	User Dashboard	12
6.4	Reading / Book Detail Page	13
6.5	Searching Page	13
6.6	Admin Dashboard	14

LIST OF TABLES

Table No.	Table Name	Page No.
4.2.1	Users Collection Structure	7
4.2.2	Books Collection Structure	7
4.2.3	CartItems Collection Structure	8
4.2.4	Otp Collection Structure	

ACRONYMS

Acronym	Full Form
MERN	MongoDB, Express.js, React, Node.js
JWT	JSON Web Token
API	Application Programming Interface
REST	Representational State Transfer
CRUD	Create, Read, Update, Delete
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
UI	User Interface
UX	User Experience
JSON	JavaScript Object Notation
MVC	Model-View-Controller
DB	Database
CSS	Cascading Style Sheets
HTML	Hypertext Markup Language
DOM	Document Object Model
NPM	Node Package Manager
ODM	Object Data Modeling
SPA	Single Page Application
CORS	Cross-Origin Resource Sharing

1. INTRODUCTION

1.1 Brief Information about the Project

The Digital Library System is a comprehensive web application designed using the modern MERN (MongoDB, Express.js, React, Node.js) stack. It replaces traditional manual library management processes with an automated digital platform. The system facilitates seamless interactions where users can easily register, log in securely, browse a wide collection of books, add items to their reading cart, and download digital copies. Simultaneously, an administrative panel is provided to manage the full catalog of resources.

1.2 Motivation and Contribution of the Project

Traditional library systems often face challenges related to physical accessibility, limited operating hours, and manual record-keeping errors. The motivation behind this project is to democratize access to knowledge by providing a 24/7 accessible digital library. By utilizing modern web technologies, this project contributes an efficient, secure, and user-friendly platform that optimizes resource management and enriches the reading experience for users through a highly interactive UI.

1.3 Objectives of the Project

- Learn and implement full-stack development using the MERN stack architecture.
- Implement secure user authentication and authorization using JSON Web Tokens (JWT) and bcrypt password hashing.
- Build a functional digital library with advanced search mechanisms and category filtering capabilities.
- Implement role-based access control to distinguish between Administrator and User permissions.
- Provide reliable book download and cart management functionality.
- Enhance practical problem-solving and software engineering technical skills.

1.4 Scope of the Project

- **User Management:** User registration and login mechanisms protected with secure JWT authentication.
- **Resource Discovery:** Book browsing interfaces enhanced with robust search algorithms and category filtering.
- **User Features:** Cart management functionality allowing users to queue and download selected books.
- **Admin Features:** A dedicated admin panel for complete book lifecycle management including CRUD operations.
- **Accessibility:** A fully responsive web interface accessible across desktop, tablet, and mobile devices.
- **Data Storage:** Utilizing MongoDB database schemas for flexible, document-based, and scalable storage.

2. SYSTEM ANALYSIS

2.1 Functional Requirements

Functional requirements outline the specific behaviors and functions the system must support. For the Digital Library System, these include:

- **User Registration and Login:** The system must allow users to create an account and authenticate.
- **JWT-based Authentication:** The system must manage sessions securely via tokens.
- **Browse and Search Books:** Users must be able to view all available books and search by title, author, or category.
- **Add Books to Cart:** Users must be able to save selected books into a digital cart.
- **Download Books:** The system must permit users to download digital book files.
- **Admin Management:** Administrators must be able to add, edit, or delete book records from the catalog.
- **Role-Based Access Control:** The system must restrict certain API routes based on user roles (Admin vs User).
- **Secure Password Storage:** The system must hash user passwords using bcrypt before saving them to the database.

2.2 Non-Functional Requirements

- **Performance:** The application should exhibit fast response times, ensuring API queries resolve under 500ms.
- **Security:** All sensitive endpoints must require valid JWT headers, and passwords must never be stored in plain text.
- **Scalability:** Utilizing MongoDB allows the database to support horizontal scaling as the user base and book catalog grow.
- **Usability:** The React UI must be intuitive, modern, and highly responsive across various screen sizes.

- **Reliability:** The backend server must gracefully handle errors to target a 99.9% uptime scenario.

2.3 Requirements Specification

2.3.1 Hardware Requirements

- **Processor:** 1 GHz or faster processor
- **RAM:** 4 GB minimum (8 GB recommended for development)
- **Disk Space:** 10 GB of free space
- **Network:** Stable Internet connection for cloud database interactions

2.3.2 Software Requirements

- **Environment:** Node.js v18.x or higher
- **Database:** MongoDB Atlas (Cloud) or local MongoDB instance
- **Browser:** Google Chrome, Mozilla Firefox, or Microsoft Edge
- **IDE:** Visual Studio Code or any equivalent code editor

3. TECHNOLOGY DESCRIPTION

3.1 Programming Languages

3.1.1 JavaScript

JavaScript is a versatile, high-level programming language that conforms to the ECMAScript specification. In this project, JavaScript serves as the foundational language across the entire stack, powering the interactive React components on the frontend as well as the server-side logic and database models on the backend via Node.js.

3.2 Frameworks and Libraries

3.2.1 React 19

React is a prominent open-source JavaScript library for building user interfaces, specifically single-page applications. React 19 introduces enhanced concurrent rendering features and hooks that allow for efficient DOM manipulation, providing a seamless browsing experience for library users.

3.2.2 Vite

Vite is a modern frontend build tool that significantly improves the development experience. By utilizing native ES modules, Vite offers exceptionally fast hot module replacement (HMR) and optimized build processes, serving as the build environment for the React application.

3.2.3 Express.js

Express.js is a minimal and flexible Node.js web application framework providing a robust set of features for web and mobile applications. It handles routing, middleware integration, and HTTP request processing, forming the backbone of the system's RESTful API.

3.2.4 bcryptjs

bcryptjs is a cryptographic library used to securely hash passwords. By applying a mathematical salt and hashing algorithm, bcrypt ensures that user passwords cannot be reverse-engineered, adding a critical layer of security to the database.

3.2.5 JSON Web Tokens (JWT)

JWT is an open standard that defines a compact and self-contained way for securely transmitting information between parties as a JSON object. It is used in this system for maintaining stateless authentication sessions and enforcing role-based access control.

3.3 Database Technology

3.3.1 MongoDB

MongoDB is a source-available, cross-platform document-oriented database program. Classified as a NoSQL database, MongoDB uses JSON-like documents with optional schemas. It provides the flexibility required to store dynamic catalog metadata and user profiles efficiently.

3.3.2 Mongoose ODM

Mongoose is an Object Data Modeling (ODM) library for MongoDB and Node.js. It manages relationships between data, provides schema validation, and translates between objects in code and the representation of those objects in MongoDB.

3.4 Tools and IDEs

3.4.1 Visual Studio Code

VS Code is a lightweight but powerful source code editor providing deep integrations for JavaScript, Node.js, and Git. It serves as the primary development environment for writing both the frontend and backend codebases.

3.4.2 Postman / MongoDB Compass

Postman is utilized for designing, testing, and documenting the REST API endpoints during development. MongoDB Compass provides a graphical user interface for visualizing database schemas, querying data, and managing collections directly.

4. DATABASE DESIGN

4.1 Database Tables (Collections)

In a MongoDB environment, traditional relational tables are replaced by collections. The Digital Library System relies primarily on three collections: Users (managing authentication and roles), Books (storing catalog metadata), and CartItems (tracking user selections).

4.2 Collection Structure

Table 4.2.1: Users Collection Structure

Column Name (Field)	Data Type	Constraint / Description
_id	ObjectId	Primary Key, Auto-generated
name	String	Required
email	String	Required, Unique constraint
password	String	Required, Hashed format
role	String	Enum: 'admin', 'user'. Default: 'user'
createdAt	Date	Default: Date.now()

Table 4.2.2: Books Collection Structure

Column Name (Field)	Data Type	Constraint / Description
_id	ObjectId	Primary Key, Auto-generated
title	String	Required
author	String	Required
category	String	Required
description	String	Optional textual description
fileUrl	String	Required, Path to downloadable file
coverImage	String	Path to the visual cover asset
downloadCount	Number	Default: 0
createdAt	Date	Default: Date.now()

4.3 Keys and Constraints

Table 4.2.3: CartItems Collection Structure

Column Name (Field)	Data Type	Constraint / Description
_id	ObjectId	Primary Key, Auto-generated
userId	ObjectId	Foreign Key referencing Users
bookId	ObjectId	Foreign Key referencing Books
addedAt	Date	Default: Date.now()

Table 4.2.4: Otp Collection Structure

Column Name (Field)	Data Type	Constraint / Description
_id	ObjectId	Primary Key
email	String	Required
otp	String	Required
createdAt	Date	Default: Date.now()
expiresAt	Date	Expiry time

The system enforces data integrity at the Mongoose schema level. The 'email' field in the Users collection employs a unique index to prevent duplicate registrations. Passwords enforce minimum length constraints before hashing. Foreign key constraints (represented as ObjectId references) link CartItems dynamically to specific Users and Books, ensuring relational queries map accurately across the NoSQL architecture.

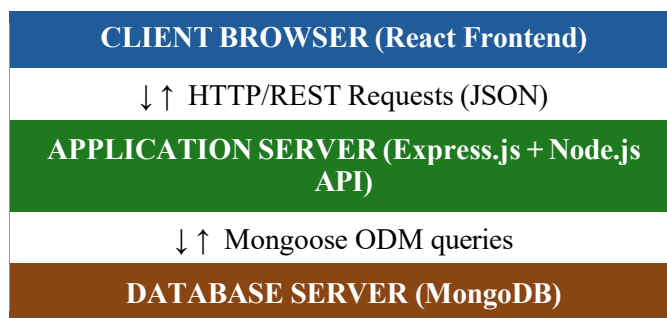
4.4 Collection Relationships

The schema design exhibits a one-to-many relationship between Users and CartItems, and a one-to-many relationship between Books and CartItems. A specific CartItem document acts as a junction entity, binding an individual User's ObjectId to a specific Book's ObjectId. This setup allows quick population mapping using Mongoose's `.populate()` method, dynamically assembling the user's cart interface.

5. SYSTEM DESIGN

5.1 System Architecture

The Digital Library System is architected based on a modern Three-Tier MVC (Model-View-Controller) pattern adapted for Single Page Applications (SPAs). The presentation tier is developed using React, rendering the UI dynamically on the client's browser. The application tier, governed by Express.js and Node.js, processes business logic, securely handles authentication via middleware, and exposes RESTful API endpoints. The data tier is supported by MongoDB, maintaining all persistent application state.



5.2 Module Descriptions

- **User Module:** Handles user profile creation, validation, and session state initialization on the frontend.
- **Login/Authentication Module:** Receives credentials, compares bcrypt hashes, and securely issues and validates JWT tokens across API boundaries.
- **Admin Module:** A restricted section accessible only to users with the 'admin' role, enabling inventory oversight, analytics viewing, and catalog manipulation.
- **Add to Cart Module:** Manages the temporary queuing of selected books, syncing local state with the backend CartItems collection.
- **Menu/Navigation Module:** A responsive routing mechanism (using React Router) that dynamically renders navbar links based on authentication status and user roles.

- **Book Management Module:** The core CRUD interface for processing book uploads, modifying metadata, handling image covers, and initiating file downloads securely.

6. SYSTEM IMPLEMENTATION

6.1 Implementation Overview

System implementation involved writing separate backend and frontend modules concurrently, allowing seamless integration via defined API endpoints. The backend was initialized first, defining Mongoose models, establishing routes, and validating JWT security middleware. Subsequently, the Vite React frontend was built, mapping styled components to application state, consuming the REST API via fetch/axios, and managing global states to reflect user interactions immediately.

6.2 User Interface Screenshots

Figure 6.1: Sign Up Page

The Sign Up page allows new users to create an account by providing their full name, email address, and password. An OTP verification step ensures secure account registration.

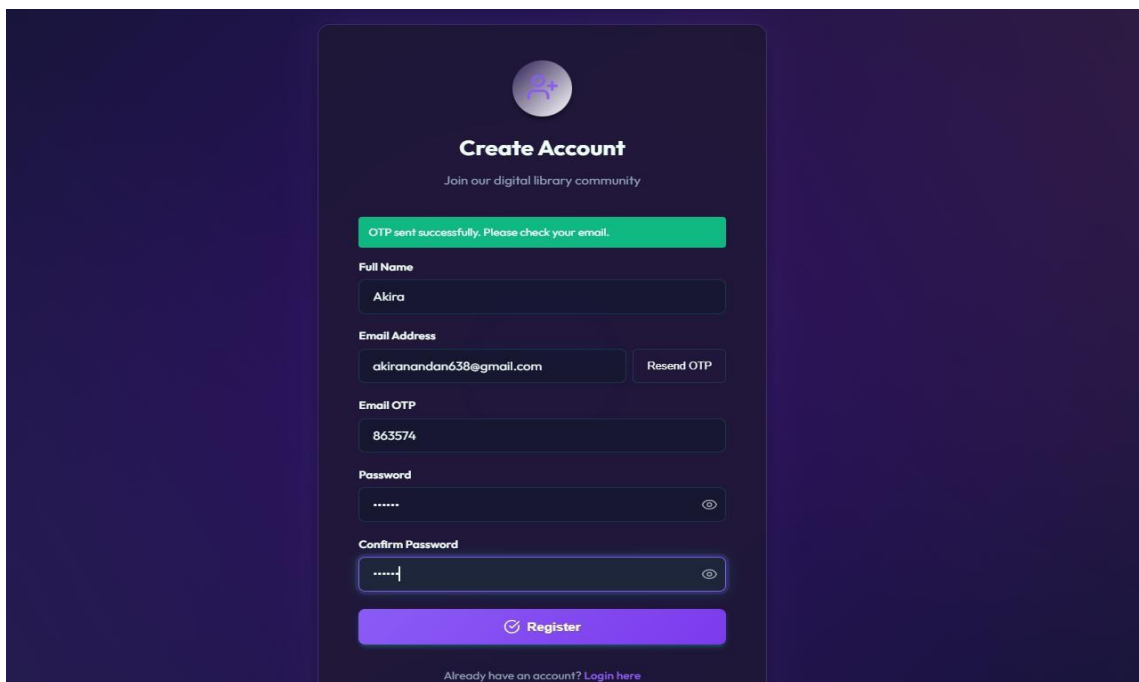


Figure 6.1: Sign Up Page

Figure 6.2: Login Page

The Login page provides a clean and secure authentication interface where registered users enter their email address and password to access the Digital Library System.

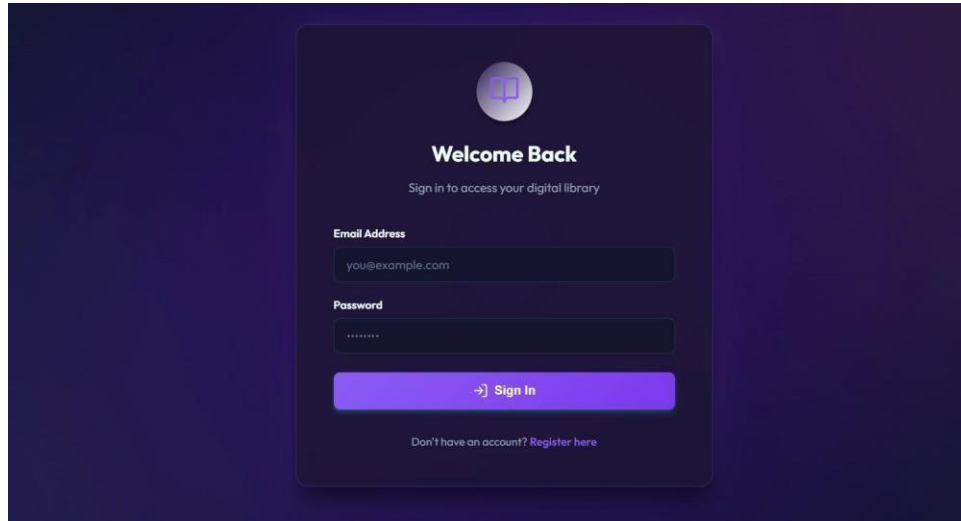


Figure 6.2: Login Page

Figure 6.3: User Dashboard

The User Dashboard displays the available book catalog with cover images, titles, and authors. Users can browse books by category and add selected titles to their reading cart.

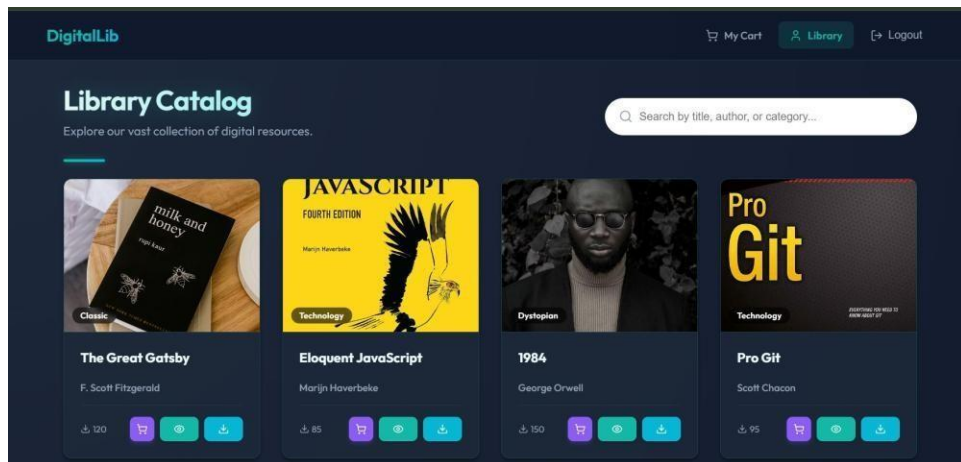


Figure 6.3: User Dashboard

Figure 6.4: Reading / Book Detail Page

The Book Detail page displays comprehensive information about a selected title including its description, author, category, and download options for the user.

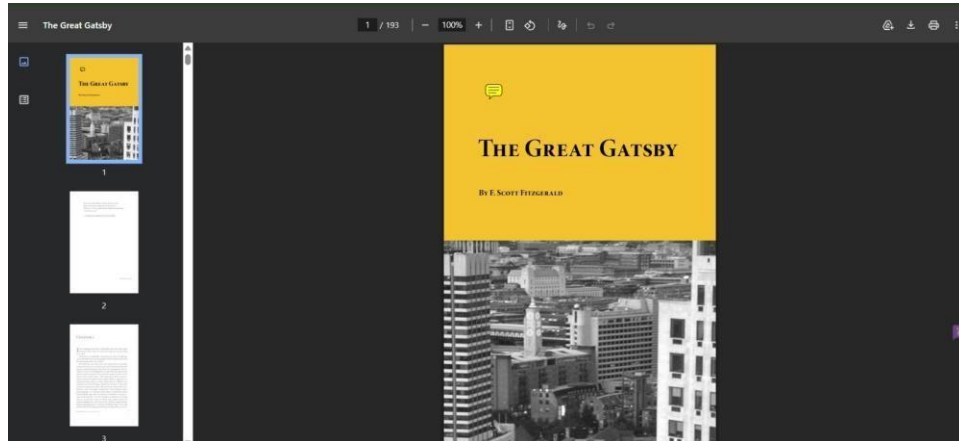


Figure 6.4: Reading / Book Detail Page

Figure 6.5: Searching Page

The Search page enables users to find books quickly by entering keywords related to the title, author, or category. Results are displayed dynamically as the user types their search query.

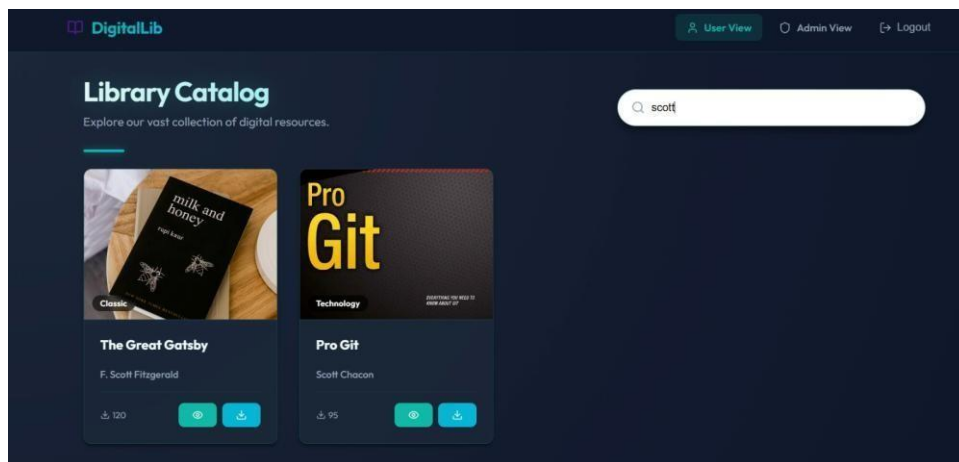


Figure 6.5: Searching Page

Figure 6.6: Admin Dashboard

The Admin Dashboard provides administrators with full control over the book catalog, including the ability to add new books, edit existing entries, view download statistics, and delete records from the system.

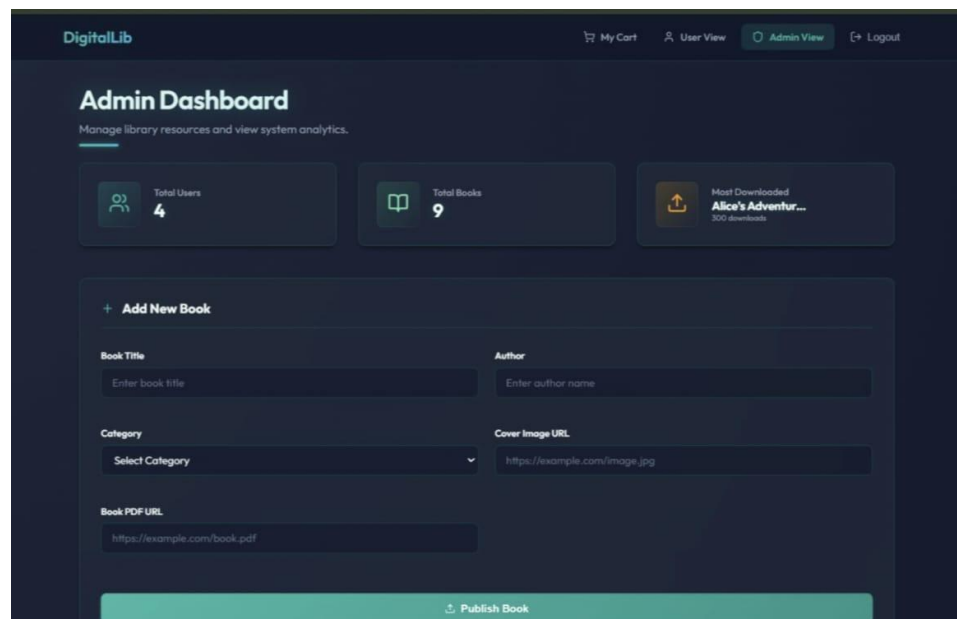


Figure 6.6: Admin Dashboard

7. CONCLUSION

The Digital Library System successfully fulfills its core objectives by delivering a highly functional, robust, and scalable MERN stack web application. By replacing inefficient traditional library tracking with a centralized digital repository, it provides significant improvements in accessibility, resource distribution, and management simplicity. The implementation of modern security practices, including JWT authentication, bcrypt password hashing, and role-based access control, ensures a secure environment for all users.

The system demonstrates proficiency in full-stack architecture by offering a smooth and intuitive user experience while successfully overcoming the limitations of manual library administration. The project also reinforced practical skills in API design, database modeling, and responsive UI development. Overall, the Digital Library System stands as an effective and production-ready solution for managing digital resources in an educational environment.

8. FUTURE ENHANCEMENTS

While the current iteration of the Digital Library System provides all foundational features, there is ample room for expansion. Future enhancements may include:

- **Mobile Application Integration:** Developing a cross-platform mobile application using React Native to cater to mobile-first users.
- **AI-Powered Recommendations:** Integrating machine learning algorithms to suggest books based on users' reading histories and preferences.
- **In-Browser PDF Viewer:** Embedding an interactive document viewer directly into the browser to eliminate the necessity of downloading files prior to reading.
- **Book Rating and Reviews:** Allowing users to rate content and leave reviews, fostering a community-driven catalog.
- **Payment Gateway Integration:** Expanding the platform to support premium or paid content by integrating third-party payment processors like Stripe.
- **Multi-Language Support:** Implementing internationalization (i18n) modules to make the library accessible to a global, non-English speaking audience.
- **Email Notifications:** Integrating automated email services (e.g., Nodemailer) for account verification, password resets, and new book alerts.
- **OAuth Login:** Permitting fast authentication via Google, GitHub, or Facebook OAuth providers to lower user onboarding friction.

9. REFERENCES

1. MongoDB Documentation. (n.d.). The MongoDB NoSQL Database. Retrieved from <https://docs.mongodb.com/>
2. Express.js Documentation. (n.d.). Fast, unopinionated, minimalist web framework for Node.js. Retrieved from <https://expressjs.com/>
3. React Official Documentation. (n.d.). A JavaScript library for building user interfaces. Retrieved from <https://react.dev/>
4. Node.js Documentation. (n.d.). Node.js v18.x API Reference. Retrieved from <https://nodejs.org/docs/>
5. JSON Web Tokens (JWT). (n.d.). Introduction to JSON Web Tokens. Retrieved from <https://jwt.io/introduction>
6. bcryptjs npm Package. (n.d.). Optimized bcrypt in JavaScript. Retrieved from <https://www.npmjs.com/package/bcryptjs>
7. Mongoose ODM Documentation. (n.d.). Elegant MongoDB object modeling for Node.js. Retrieved from <https://mongoosejs.com/docs/>
8. MDN Web Docs. (n.d.). JavaScript, HTML, and CSS References. Retrieved from <https://developer.mozilla.org/>

10. APPENDIX

A.1 Sample Code – Backend Authentication Route

```
// routes/auth.js

const express = require('express');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const User = require('../models/User');
const router = express.Router();

router.post('/login', async (req, res) => {
  try {
    const { email, password } = req.body;
    const user = await User.findOne({ email });
    if (!user) return res.status(404).json({ message: 'User not found' });
    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) return res.status(400).json({ message: 'Invalid credentials' });
    const token = jwt.sign(
      { id: user._id, role: user.role },
      process.env.JWT_SECRET,
      { expiresIn: '1d' }
    );
    res.status(200).json({ token, user: { id: user._id, name: user.name, role: user.role } });
  } catch (error) {
    res.status(500).json({ message: 'Server error' });
  }
});

module.exports = router;
```

A.2 Sample Code – Frontend React Component

```
// components/BookCard.jsx
import React from 'react';
import { useNavigate } from 'react-router-dom';

const BookCard = ({ book, onAddToCart }) => {
  const navigate = useNavigate();
  return (
    <div className='book-card border p-4 shadow-sm'>
      <img src={book.coverImage} alt={book.title} className='w-full h-48 object-cover' />
      <h3 className='text-lg font-bold mt-2'>{book.title}</h3>
      <p className='text-sm text-gray-600'>By {book.author}</p>
      <div className='flex justify-between mt-4'>
        <button onClick={() => navigate(`/books/${book._id}`)}
          className='bg-blue-500 text-white px-3 py-1 rounded'>
          View Details
        </button>
      </div>
    </div>
  );
};
```

```
        </button>
        <button onClick={() => onAddToCart(book._id)}
          className='bg-green-500 text-white px-3 py-1 rounded'>
          Add to Cart
        </button>
      </div>
    </div>
  );
};

export default BookCard;
```